

Programming Assignment 4—Static Single Assignment and Redundant Phi Node Removal

Marc Lin

1 Static Single Assignment

In the SSA part, the most important thing is how to implement functions in sections 2.2 (from Braun's paper). For *readVariableRecursive()*, we separate phi node insertion to 3 cases: 1) incomplete, 2) single predecessor, 3) multiple predecessors with sealed. We only insert phi node in case 1) and 3).

While doing insertion, I refactored the check for *insertPoint* because *getFirstNonPHI()* return end iterator of that block instead of *nullptr*. If there's no non-PHI instructions, append new phi node at the end of BB, else insert before instructions in BB.

```
// incomplete CFG
if (!mSealedBlocks.count(block)) {
    // 1: Type of var, 2: preserved basic blocks before, 3: name, 4: insertion point of phi
    llvm::PHINode* phi = nullptr;
    // 1) when getFirstNonPHI returns end iterator
    if (insertPoint == block->end()) {
        // give block iterator as the argument to create PHINode
        // (NO non-PHI instructions --> append the new PHI node at the end of basic block)
        phi = llvm::PHINode::Create(Ty, 0, "Phi", block);
    } // 2) when getFirstNonPHI NOT returns end iterator
    else {
        // give instruction iterator instead
        phi = llvm::PHINode::Create(Ty, 0, "Phi", insertPoint);
    }
    retVal = phi; // upcasting
    (*mIncompletePhis[block])[var] = phi;
}
```

For *tryRemoveTrivialPhi()*, this is corresponding to the trivial definition described in paper. First, traverse all operands of phi node and check if it's trivial, non-trivial or unreachable. After that, we do recursive removal, so we must find all the nodes that are using this phi node (i.e. users). Check these nodes to see if they become phi node or not, by using *dyn_cast*. At the end, do redirection by *replaceAllUsesWith* and update *mVarDefs*.

```

// save all the users first, then remove
// Remember all users except the phi itself
std::vector<llvm::PHINode*> phiUsers;
for (auto UI = phi->user_begin() ; UI != phi->user_end() ; UI++) {
    llvm::User *user = *UI;
    if (user != phi) {
        // only collected when user is a PHINode
        if (llvm::PHINode* usePhi = llvm::dyn_cast<llvm::PHINode>(user)) {
            phiUsers.push_back(usePhi);
        }
    }
}

// Reroute all uses of phi to same and remove phi
// must update the variable definition map from "phi" to use "same"
phi->replaceAllUsesWith(same);
BasicBlock *block = phi->getParent();
if (mVarDefs.count(block)) {
    for (auto &p : *mVarDefs[block]) { // SubMap*
        if (p.second == phi) {
            p.second = same;
        }
    }
}
// erase the replaced phi node
phi->eraseFromParent();

// Try to recursively remove all phi users, which might have become trivial
for (auto use : phiUsers) {
    // already done the casting before
    tryRemoveTrivialPhi(use);
}
return same;

```

2 Redundant Phi Node Removal

To remove SCCs that are related to redundant phi node, first add all phi nodes to build induced subgraph, also add edges **between phi nodes**. `getSCCs()` is already given so we can directly get all the SCCs.

```

// 1. build induced graph
// (add node)
for (auto &phi : phiFunctions) {
    graph.getOrAddToGraph(phi); // upcasting: PHINode* -> Value*
}
// (add egde)
for (auto &phi : phiFunctions) {
    GraphNode *src = graph.getOrAddToGraph(phi);
    // only make egde in SCC iff dest is also in set P (Lemma 1)
    for (unsigned i = 0 ; i < phi->getNumIncomingValues() ; i++) {
        Value *op = phi->getIncomingValue(i);
        if (PHINode *dest = dyn_cast<PHINode>(op)) {
            if (phiFunctions.count(dest)) {
                src->addEdge(graph.getOrAddToGraph(dest)); // find dest in Graph, then add edge (src, dest)
            }
        }
    }
}

```

For each SCC, we use *processSCC()* to decide whether the whole SCC is redundant or not. Every node in SCC is categorized into 1) inner, point to other phi nodes in SCC, 2) outer, point to operands outside of SCC.

```
for (auto &v : scc) { // Value * (but actually they are all phi node)
    PHINode* phi = dyn_cast<PHINode>(v); // cast to phi node
    bool isInner = true;
    for (unsigned i = 0 ; i < phi->getNumIncomingValues() ; i++) {
        Value *op = phi->getIncomingValue(i);
        if (!scc.count(op)) { // operand not in scc
            outerOps.insert(op);
            isInner = false;
        }
    }
    if (isInner) {
        inner.insert(phi);
    }
}
```

If there is only 1 outer operand, then the whole SCC is redundant (Lemma 1).

Replace all the usage by this phi to the outer value *s*.

If there are more than 1 outer operand, recursively call *removeRedundantPhis()* on *inner*, try to find redundant nodes in a smaller sacle.

```
if (outerOps.size() == 1) {
    llvm::Value* uniqueVal = *outerOps.begin(); // outerOps.pop()
    replaceSCCByValue(scc, uniqueVal, phiFunctions);
}
else if (outerOps.size() > 1) {
    removeRedundantPhis(inner);
}
```

I implemented *replaceSCCByValue()* by following the flow

“*replaceAllUsesWith(s) -> erase(phi) -> eraseFromParent()*”.

```
void RedundantPhiRemoval::replaceSCCByValue(std::set<Value*>& scc, Value* val, std::set<PHINode*>& phiFunctions) {
    for (Value *v : scc) {
        PHINode* phi = dyn_cast<PHINode>(v);
        phi->replaceAllUsesWith(val);
        phiFunctions.erase(phi); // avoid operation in removeRedundantPhis read the unused memory
        phi->eraseFromParent();
    }
}
```

3 Local Copy Propagation After SSA

Only arrays remain as memory operations (via GEP + store/load) after we complete SSA form. Following the pseudocode in instructions, I figured out *avail* should be a map, with `std::pair<llvm::Value*, llvm::Value*>` as key and `llvm::Value*` as value. Where *base* is array head address, *index* is offset, and *val* is the actual value assigned to array. Therefore, the most important part is how I extracted these values from *inst*. $\{base, index\}$ should be extracted from *GEP* pointer, while *val* should be extracted from *StoreInst* since we use store to assign value and use *GetElementPtrInst* to do pointer address arithmetic.

```
for (auto &inst : bb) {
    // if Inst is a store to a single-index GetElementPtr
    // StoreInst ex. ====> store i32 %val, i32* %ptr
    // GEP ex. ====> %ptr = getelementptr <type>, <base>, <index 0>, <index 1>, ...
    if (auto *SI = dyn_cast<StoreInst>(&inst)) {
        if (auto *GEP = dyn_cast<GetElementPtrInst>(SI->getPointerOperand())) {
            Value *base = GEP->getPointerOperand();
            Value *index = GEP->getOperand(GEP->getNumOperands() - 1); // index operand is the last operand
            Value *val = SI->getValueOperand();

            if (!isa<ConstantInt>(index)) {
                for (auto it = avail.begin(); it != avail.end(); ) {
                    if ((it->first).first == base) {
                        it = avail.erase(it); // return the following iterator
                    } else {
                        it++; // when not removing, move to next iterator
                    }
                }
            }

            avail[{base, index}] = val;
        }
    }
}
```

After finishing scan of *LoadInst* and *StoreInst*, remove redundant load. Also erase the associated *GetElementPtr* if it becomes unused. The pass not only propagates stored values forward but also cleans up the unnecessary instructions left behind.

```
for (auto *D : dead) { // LoadInst*
    Value *G = D->getPointerOperand();
    D->eraseFromParent();
    if (auto *GEP = dyn_cast<GetElementPtrInst>(G)) {
        if (GEP->use_empty()) {
            GEP->eraseFromParent();
        }
    }
}
```